SURVEY 4 + 5

Python OOP Assessment

Bootcamp assessment Total marks: **100**

Name: _____ Student ID: _____

Instructions

- Answer **both questions**. Use Python 3 and the standard library only. Submit one runnable file per question: `question_1.py` and `question_2.py`.
- Use `__init__()` to initialize objects. Write reusable classes and a driver section that demonstrates the required behavior.
- **Do not use `@property` or property setters.** Use ordinary methods such as `set_email()`, `show_balance()`, and `is_available()`. Use `@classmethod`, `@staticmethod`, and `@abstractmethod` where required.
- Unless a different failure result is specified, reject invalid input with `ValueError` before changing state. Catch expected errors in the driver so later demonstrations still run.
- This is an in-memory console assessment. No database, web framework, interface, real payment service, or real email delivery is required.

Question 1: Campus Services Manager

40 MARKS

Build a small campus model that manages student results, student wallets, library books, and staff salaries. Implement the following classes in the same program.

(a) Student results [8 marks]

Create a `Student` class with `name`, `student_id`, and `marks`.

- Initialize all three attributes. Accept numeric marks from 0 to 100, inclusive; reject marks outside this range.
- Implement `get_grade()` using the exact rules below.
- Implement `__str__()` to return the student's name, ID, marks, and grade in a readable summary.

A	B	C	D	F
$80 \leq m \leq 100$	$70 \leq m < 80$	$60 \leq m < 70$	$50 \leq m < 60$	$0 \leq m < 50$

(b) Student wallet [8 marks]

Create `BankAccount(account_holder, opening_balance)`. Set its class variable `bank_name` to `"Campus Bank"`.

- Keep the balance in an internal attribute such as `_balance` or `__balance`. Reject a negative opening balance.
- Implement `deposit(amount)`, `withdraw(amount)`, and `show_balance()`. Both transaction amounts must be positive; a withdrawal cannot exceed the current balance.
- Return the current balance from `show_balance()`. Application code must change the balance only through the transaction methods.

Question 1: Campus Services Manager

Continued

(c) Staff hierarchy [8 marks]

Create an `Employee` base class and two child classes.

- `Employee(name, base_salary)` stores the inherited data. Its `calculate_salary()` returns the base salary, and `summary()` returns a readable name, role, and calculated salary.
- `Developer(name, base_salary, bonus)` uses `super()` to initialize inherited data. Override `calculate_salary()` to return base salary plus bonus.
- `Manager(name, base_salary, allowance)` also uses `super()`. Override `calculate_salary()` to return base salary plus allowance.
- Reject negative salary, bonus, or allowance values. Make `summary()` use `self.calculate_salary()` so it reflects each role. Put different employee objects in one list and print each object's summary using the same loop.

(d) Book collection and borrowing [10 marks]

Create `Book(title, author)` and `Library()`.

- Each book starts available. Keep availability in an internal attribute. Implement `is_available()`, `borrow()`, and `return_book()`.
- `borrow()` returns `True` and makes an available book unavailable. It returns `False` if the book is already borrowed. `return_book()` performs the reverse change and returns `False` if the book was already available.
- Implement `Book.__str__()` to show the title, author, and availability. The library and driver must use book methods to change availability.
- `Library` stores a list of `Book` objects. Its methods are `add_book(book)`, `borrow_book(title)`, `return_book(title)`, and `list_available_books()`.
- Use unique, case-sensitive titles. Reject duplicate titles. Borrowing or returning an unknown title returns `False`; otherwise return the result of the corresponding book method. `list_available_books()` returns a list of available book objects.

Borrower records, due dates, fines, and wallet deductions for borrowing are outside the scope of this question.

(e) Demonstration and design comments [6 marks]

Write a driver that creates at least two students, one wallet using a student's name, two books, a library, and one object of each employee class. Demonstrate the cases below, including the rejected operations.

Demonstration	Expected behavior
Student marks 85 and 47; also check 50, 60, 70, 80	Grades A and F; boundary grades D, C, B, A.
Opening balance 1000; deposit 500; withdraw 300	Final balance is 1200.
Try a zero deposit, a negative withdrawal, and an overdraw	Each is rejected; balance remains unchanged.
Borrow the same book twice, then return it	Results are True, False, True; availability updates.
Developer: base 30000, bonus 5000; Manager: base 40000, allowance 10000	Calculated salaries are 35000 and 50000.

Add brief comments identifying **encapsulation**, **inheritance**, and **polymorphism** in your program. Driver and expected behavior: 4 marks; design comments: 2 marks.

Question 2: Online Bootcamp Store

60 MARKS

Build an in-memory store that registers users, manages products and stock, creates orders, and processes simulated payments. Keep responsibilities in separate classes and connect the objects through the relationships specified below. This question is independent of Question 1.

(a) User accounts and roles [12 marks]

Create `User(username, email, password)`.

- Provide a static method `validate_username(username)` that returns `True` only for 3–20 characters containing letters, digits, or underscores, with no spaces. Reject invalid usernames in the constructor.
- Implement the ordinary methods `set_email(email)` and `get_email()`. A valid email has exactly one `@`, a non-empty part before it, and a domain containing a dot that is neither its first nor its last character. Reject whitespace. Apply the same validation during construction and later updates.
- Keep the password in an internal attribute such as `_password` or `__password`. Require a non-empty string. `check_password(password)` and `login(password)` return a Boolean indicating whether the supplied password matches.
- Implement `change_password(old_password, new_password)`. Return `False` without changing anything if the old password is wrong. If it matches, reject an empty new password; otherwise update the password and return `True`.
- Maintain a shared class variable `total_users`. Count each successfully constructed `User`, `Customer`, or `Admin` exactly once. Provide `@classmethod get_total_users()` to return that shared count. Failed construction must not increase it.
- Implement `__str__()` to show username, email, and role. Never include the password. Base `User.permissions()` returns an empty list.

Create `Customer` and `Admin` subclasses. Use `super()` in their constructors and override `permissions()`: a customer returns `["browse", "place_order"]`; an admin returns `["browse", "manage_products", "view_orders"]`. Write one function `show_permissions(user)` that prints permissions for either role through the same method call. Permission enforcement is not required.

Practice-only account model: Use dummy passwords. This exercise demonstrates OOP; real applications must store secure password hashes rather than plaintext passwords.

(b) Products and inventory [10 marks]

Create `Product(name, price, quantity)` and `Inventory()`.

- Require a non-empty name, a positive numeric price, and a non-negative integer stock quantity. Keep stock in an internal attribute.
- Implement `get_quantity()` and `reduce_stock(amount)`. A reduction must be a positive integer and cannot exceed available stock. Implement `__str__()` to show name, price, and remaining quantity.
- `Inventory` stores a list of product objects. Implement `add_product(product)`, `show_products()`, `find_product(name)`, and `total_inventory_value()`.
- Reject duplicate, case-sensitive names. `show_products()` prints the products; `find_product(name)` returns the matching object or `None`. Inventory value is the sum of each price multiplied by its remaining stock.

Question 2: Online Bootcamp Store

Continued

(c) Interchangeable payment methods [10 marks]

Use the `abc` module to create an abstract `PaymentMethod` class with an abstract `pay(amount)` method.

- Implement `CardPayment`, `MobileWalletPayment`, and `CashPayment`. Each `pay(amount)` rejects a non-positive amount, prints a message naming its payment method and the amount, and returns `True` to simulate success.
- Write a common function `checkout(payment_method, amount)`. Call `pay(amount)` on the supplied payment object and return its result.
- Demonstrate this function with all three payment types. Do not branch on class names or use `type()` / `isinstance()` to choose a payment implementation.

Do not request card numbers, wallet credentials, or real financial information.

(d) Orders and object relationships [14 marks]

Create `OrderItem(product, quantity)` and `Order(customer)`.

OrderItem

- Reference an existing `Product` and require a positive integer quantity. Store the product's current price as `unit_price` when the item is created. `subtotal()` returns unit price multiplied by quantity.
- Products are created separately and can be referenced by multiple order items. Product prices stay unchanged during the demonstration.

Order

- Reference an existing `Customer`; start with an empty item list and an unpaid state. Create and own `OrderItem` objects internally through `add_item(product, quantity)`.
- Require a positive integer quantity. For this assessment, add each product at most once per order; reject a second line for the same product. Reject quantities exceeding current stock. Adding an item does not reserve or reduce stock.
- Implement `total()` as the sum of all item subtotals.
- Implement `checkout(payment_method)`. Reject an empty order or an already-paid order, then recheck the stock for **every** item before attempting payment. If any item lacks stock, do not call the payment method or change any stock.
- Call the common `checkout(payment_method, amount)` function from part (c), using the order total. On `True`, reduce stock through product methods, mark the order paid, and return `True`. On `False`, return `False` without changing stock or paid state.
- Reject attempts to add items after successful payment. Repeating checkout must not charge again or deduct stock again.

Relationship requirements: Composition – an order creates and owns its order items. **Association** – an order references an existing customer, and each order item references an existing product. **Polymorphism** – an order accepts any payment implementation through the common payment interface. Add short comments beside the relevant code.

Process orders sequentially. Discounts, taxes, shipping, refunds, concurrent purchases, and stock reservation are outside the scope of this question.

Question 2: Online Bootcamp Store

Continued

(e) Registration service and dependency injection [10 marks]

Create `UserRepository`, `EmailSender`, and `UserService`.

- `UserRepository` stores users in memory. Implement `save(user)` and `find_by_username(username)`. Reject duplicate, case-sensitive usernames; a failed lookup returns `None`.
- `EmailSender.send_welcome(user)` prints a welcome message containing the user's username and email address.
- Initialize the service as `UserService(repo, mailer)` and store those supplied dependencies. Do not construct a repository or email sender inside the service.
- Implement `register(username, email, password)`. Check for a duplicate username before creating an object. Then create a `Customer`, save it through the repository, send the welcome message through the email sender, and return that customer.
- Invalid or duplicate registration must not save a user, send a welcome message, or increase `User.total_users`. Assume the in-memory save and simulated email operations succeed for valid input.

Use the customer returned by registration when creating the demonstration order. Add a short comment identifying **dependency injection**.

(f) End-to-end demonstration [4 marks]

Create the repository and email sender outside the service, then register two customers through the service and create one admin directly. Use dummy passwords and demonstrate the following:

Area	Required evidence
Accounts and roles	Print readable users and permissions for both roles. Show a successful login, a failed login, a rejected password change, and a successful change. Confirm only the new password works after the change.
Validation and counts	Starting from zero users, two registered customers plus one admin give a count of 3. Try an invalid username, an invalid email, a duplicate registration, and an invalid email update; verify that existing state and user count stay unchanged.
Inventory	Add Keyboard (price 2500, stock 4) and Mouse (price 1200, stock 6). Display both and their initial total value of 17200. An unknown product lookup returns None.
Successful order	Order 2 keyboards and 1 mouse for a registered customer. Total: 6200. After one successful checkout, remaining stocks are 2 and 5; inventory value is 11000.
Rejected operations	Demonstrate a negative product price, a negative stock quantity, an excessive stock request, an empty checkout, and a repeated checkout. Show that rejected operations leave state unchanged.
Payment flexibility	Demonstrate the common checkout function with card, mobile wallet, and cash objects. Print the payment result for each.

Submission checklist

Submit `question_1.py`, `question_2.py`, and `output.txt` containing the console output of both drivers. Keep relationship comments beside the relevant code. Use clear object names and catch expected validation errors. Exact wording of printed messages is flexible; values and behavior must match the requirements.

Mark allocation: Question 1: 40 marks Question 2: 60 marks Total: 100 marks